

操作系统实验报告

姓名：李宗杭 学号：2311451 学院：软件学院

实验日期：2025.11.26 星期三 11-12 节

实验题目：生产者消费者问题

一、实验内容

编写一个 c/c++ 程序，实现 I 个生产者 J 个消费者问题，其中共享缓冲区的大小为 N，所有生产者共生产 K (K>N) 个产品后结束，所有消费者共消费 K 个产品后结束。

二、实现步骤

1、环境准备

安装/更新核心编译工具

```
sudo apt-get install -y build-essential
```

2、实现细节

(1) 日志内容

角色	内容
生产者	开始生产，预计耗时time秒
	生产产品item完成（校验码：checksum）
	将产品item存入缓冲区buffer
	所有产品已生产完毕，退出线程
	准备进入临界区
	已进入临界区
	已离开临界区
消费者	开始消费产品item，预计耗时time秒
	消费产品item完成（校验码：checksum）
	从缓冲区buffer取出产品item（生产者：producer，生产时间：time）
	所有产品已消费完毕，退出线程
	准备进入临界区
	已进入临界区
	已离开临界区

(2) 同步机制

信号量：控制缓冲区空/满状态；

互斥量：保护缓冲区操作。

(3) 数据结构：链表。

3、代码编写（注释详细）

(1) 头文件引入与全局变量

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <pthread.h>
4  #include <semaphore.h>
5  #include <time.h>
6  #include <unistd.h>
7  #include <stdint.h>
8  #include <stdbool.h>
9
10 // ===== 全局变量 =====
11
12 // 参数配置
13 #define PRODUCER_NUM 2          // I: 生产者线程数
14 #define CONSUMER_NUM 4         // J: 消费者线程数
15 #define BUFFER_SIZE 5          // N: 缓冲区大小
16 #define TOTAL_PRODUCT 10       // K: 总产品数
17 #define DATA_SIZE 256         // 产品数据长度
18
19 // 产品结构体
20 typedef struct item_st {
21     int id;                      // 产品编号 (1~TOTAL_PRODUCT)
22     time_t timestamp_p;          // 生产时间 (时间戳)
23     int produce_id;              // 生产者编号 (1~PRODUCER_NUM)
24     uint32_t checksum;           // 产品校验码 (简单计算: id + 生产时间)
25     uint32_t data[DATA_SIZE];    // 产品详细信息 (随机填充)
26 } ITEM_ST;
27
28 // 缓冲区节点结构体
29 typedef struct buffer_st {
30     int id;                      // 缓冲区编号 (1~BUFFER_SIZE)
31     bool isempty;                // 是否为空
32     ITEM_ST item;                // 存储的产品
33     struct buffer_st* nextbuf;   // 下一个缓冲区节点
34 } BUFFER_ST;
35
36 // 全局变量 (共享资源)
37 BUFFER_ST* buffer_head = NULL;  // 缓冲区链表头
38 int produced_count = 0;         // 已生产产品数
39 int consumed_count = 0;         // 已消费产品数
40 int product_id_count = 1;       // 产品编号计数器
41
42 // 同步机制: 信号量 + 互斥量
43 sem_t empty_sem;                // 空缓冲区信号量 (初始BUFFER_SIZE)
44 sem_t full_sem;                 // 满缓冲区信号量 (初始0)
45 pthread_mutex_t mutex;          // 互斥锁
```

(2) 工具函数：计算产品校验码

```
49 // 计算产品校验码
50 uint32_t calculate_checksum(ITEM_ST* item) {
51     // 校验码 = 产品编号 + 生产时间戳
52     return (uint32_t)(item->id + item->timestamp_p);
53 }
```

(3) 工具函数：获取格式化时间

```
55 // 获取当前时间字符串
56 void get_current_time(char* time_str, int len) {
57     time_t now = time(NULL);
58     strftime(time_str, len, "%Y-%m-%d %H:%M:%S", localtime(&now));
59 }
```

(4) 工具函数：创建新产品

```
61 // 创建新产品
62 ITEM_ST create_product(int producer_id) {
63     ITEM_ST new_item;
64
65     // 分配产品编号
66     pthread_mutex_lock(&mutex);
67     new_item.id = product_id_count++;
68     pthread_mutex_unlock(&mutex);
69
70     // 设置产品属性
71     new_item.timestamp_p = time(NULL);
72     new_item.produce_id = producer_id;
73
74     // 填充产品数据
75     for (int i = 0; i < DATA_SIZE; i++) {
76         new_item.data[i] = rand() % 1000;
77     }
78
79     // 计算校验码
80     new_item.checksum = calculate_checksum(&new_item);
81
82     return new_item;
83 }
```

(5) 工具函数：生产产品

```
85 // 生产产品
86 ITEM_ST produce_product(int producer_id) {
87     char time_str[32];
88
89     // 随机生产时间，模拟生产耗时
90     int produce_time = rand() % 3 + 1; // 1-3秒
91     get_current_time(time_str, sizeof(time_str));
92     printf("[%s] 生产者%d: 开始生产, 预计耗时%d秒\n",
93         time_str, producer_id, produce_time);
94
95     sleep(produce_time);
96     ITEM_ST new_item = create_product(producer_id);
97
98     get_current_time(time_str, sizeof(time_str));
99     printf("[%s] 生产者%d: 生产产品%d完成 (校验码: %u)\n",
100         time_str, producer_id, new_item.id, new_item.checksum);
101
102     return new_item;
103 }
```

(6) 工具函数：消费产品

```
105 // 消费产品
106 void consume_product(ITEM_ST* item, int consumer_id) {
107     char time_str[32];
108
109     // 随机消费时间，模拟消费耗时
110     int consume_time = rand() % 3 + 1; // 1-3秒
111     get_current_time(time_str, sizeof(time_str));
112     printf("[%s] 消费者%d: 开始消费产品%d, 预计耗时%d秒\n",
113         time_str, consumer_id, item->id, consume_time);
114
115     sleep(consume_time);
116
117     get_current_time(time_str, sizeof(time_str));
118     printf("[%s] 消费者%d: 消费产品%d完成 (校验码: %u)\n",
119         time_str, consumer_id, item->id, item->checksum);
120 }
```

(7) 工具函数：将产品存入缓冲区

```
123 // 将产品存入缓冲区
124 void store_product_to_buffer(ITEM_ST* item, int producer_id) {
125     char time_str[32];
126
127     // 找到第一个空缓冲区
128     BUFFER_ST* temp = buffer_head;
129     while (temp != NULL && !temp->isempty) {
130         temp = temp->nextbuf;
131     }
132
133     // 存入产品
134     if (temp != NULL) {
135         temp->item = *item;
136         temp->isempty = false;
137         get_current_time(time_str, sizeof(time_str));
138         printf("[%s] 生产者%d: 将产品%d 存入缓冲区%d\n",
139             time_str, producer_id, item->id, temp->id);
140     }
141 }
```

(8) 工具函数：从缓冲区取出产品

```
143 // 从缓冲区取出产品
144 ITEM_ST fetch_product_from_buffer(int consumer_id) {
145     char time_str[32];
146     char produce_time_str[32];
147
148     // 找到第一个非空缓冲区
149     BUFFER_ST* temp = buffer_head;
150     while (temp != NULL && temp->isempty) {
151         temp = temp->nextbuf;
152     }
153
154     // 取出产品
155     ITEM_ST consumed_item = temp->item;
156     temp->isempty = true;
157
158     get_current_time(time_str, sizeof(time_str));
159     strftime(produce_time_str, sizeof(produce_time_str),
160             "%Y-%m-%d %H:%M:%S", localtime(&consumed_item.timestamp_p));
161     printf("[%s] 消费者%d: 从缓冲区%d 取出产品%d (生产者: %d, 生产时间: %s)\n",
162           time_str, consumer_id, temp->id, consumed_item.id,
163           consumed_item.produce_id, produce_time_str);
164
165     return consumed_item;
166 }
```

(9) 工具函数：初始化缓冲区

```
168 // 初始化缓冲区链表: 创建BUFFER_SIZE个节点
169 void init_buffer() {
170     BUFFER_ST* prev = NULL;
171     for (int i = 1; i <= BUFFER_SIZE; i++) {
172         BUFFER_ST* new_buf = (BUFFER_ST*)malloc(sizeof(BUFFER_ST));
173         if (new_buf == NULL) {
174             perror("malloc buffer failed");
175             exit(1);
176         }
177
178         // 初始化节点属性
179         new_buf->id = i;
180         new_buf->isempty = true;
181         new_buf->nextbuf = NULL;
182
183         // 链接到链表
184         if (prev == NULL) {
185             buffer_head = new_buf;
186         } else {
187             prev->nextbuf = new_buf;
188         }
189         prev = new_buf;
190     }
191     printf("缓冲区初始化完成: 共%d个节点\n", BUFFER_SIZE);
192 }
```

(10) 线程函数：生产者线程

```
196 // 生产者线程函数: arg为生产者编号
197 void* producer_thread(void* arg) {
198     int producer_id = *(int*)arg;
199     char time_str[32];
200
201     while (1) {
202         // 1. 检查是否已生产完所有产品
203         pthread_mutex_lock(&mutex);
204         if (produced_count >= TOTAL_PRODUCT) {
205             get_current_time(time_str, sizeof(time_str));
206             printf("[%s] 生产者%d: 所有产品已生产完毕, 退出线程\n", time_str, producer_id);
207             pthread_mutex_unlock(&mutex); // 退出前必须解锁
208             pthread_exit(NULL);
209         }
210         else {
211             produced_count++;
212         }
213         pthread_mutex_unlock(&mutex);
214
215         // 2. 生产产品
216         ITEM_ST new_item = produce_product(producer_id);
217
218         // 3. P操作获取空缓冲区
219         sem_wait(&empty_sem);
220
221         // 4. 进入临界区, 存入产品
222         get_current_time(time_str, sizeof(time_str));
223         printf("[%s] 生产者%d: 准备进入临界区\n", time_str, producer_id);
224         pthread_mutex_lock(&mutex);
225         get_current_time(time_str, sizeof(time_str));
226         printf("[%s] 生产者%d: 已进入临界区\n", time_str, producer_id);
227
228         store_product_to_buffer(&new_item, producer_id);
229
230         get_current_time(time_str, sizeof(time_str));
231         printf("[%s] 生产者%d: 已离开临界区\n", time_str, producer_id);
232         pthread_mutex_unlock(&mutex);
233
234         // 5. V操作通知新产品
235         sem_post(&full_sem);
236     }
237 }
```

(11) 线程函数：消费者线程

```
239 // 消费者线程函数: arg为消费者编号
240 void* consumer_thread(void* arg) {
241     int consumer_id = *(int*)arg;
242     char time_str[32];
243
244     while (1) {
245         // 1. 检查是否已消费完所有产品
246         pthread_mutex_lock(&mutex);
247         if (consumed_count >= TOTAL_PRODUCT) {
248             get_current_time(time_str, sizeof(time_str));
249             printf("[%s] 消费者%d: 所有产品已消费完毕, 退出线程\n", time_str, consumer_id);
250             pthread_mutex_unlock(&mutex); // 退出前必须解锁
251             pthread_exit(NULL);
252         }
253         else {
254             consumed_count++;
255         }
256         pthread_mutex_unlock(&mutex);
257
258         // 2. P操作等待新产品
259         sem_wait(&full_sem);
260
261         // 3. 进入临界区, 取出产品
262         get_current_time(time_str, sizeof(time_str));
263         printf("[%s] 消费者%d: 准备进入临界区\n", time_str, consumer_id);
264         pthread_mutex_lock(&mutex);
265         get_current_time(time_str, sizeof(time_str));
266         printf("[%s] 消费者%d: 已进入临界区\n", time_str, consumer_id);
267
268         int buffer_id;
269         ITEM_ST consumed_item = fetch_product_from_buffer(consumer_id);
270
271         get_current_time(time_str, sizeof(time_str));
272         printf("[%s] 消费者%d: 已离开临界区\n", time_str, consumer_id);
273         pthread_mutex_unlock(&mutex);
274
275         // 4. V操作通知空缓冲区
276         sem_post(&empty_sem);
277
278         // 5. 消费产品
279         consume_product(&consumed_item, consumer_id);
280     }
281 }
```

(12) 主函数

```
283 // ===== 主函数 =====
284
285 int main() {
286     // 1. 初始化
287     srand((unsigned int)time(NULL));
288     char time_str[32];
289     printf("=====\n");
290     printf("生产者消费者实验: 生产者%d个, 消费者%d个, 缓冲区大小%d, 总产品%d个\n",
291           PRODUCER_NUM, CONSUMER_NUM, BUFFER_SIZE, TOTAL_PRODUCT);
292     printf("=====\n");
293
294     // 初始化缓冲区链表
295     init_buffer();
296
297     // 初始化信号量和互斥量
298     if (sem_init(&empty_sem, 0, BUFFER_SIZE) == -1) {
299         perror("sem_init empty_sem failed");
300         exit(1);
301     }
302     if (sem_init(&full_sem, 0, 0) == -1) {
303         perror("sem_init full_sem failed");
304         exit(1);
305     }
306     if (pthread_mutex_init(&mutex, NULL) != 0) {
307         perror("pthread_mutex_init failed");
308         exit(1);
309     }
310
311     // 2. 创建生产者线程
312     pthread_t producer_tids[PRODUCER_NUM];
313     int producer_ids[PRODUCER_NUM];
314     for (int i = 0; i < PRODUCER_NUM; i++) {
315         producer_ids[i] = i + 1;
316         if (pthread_create(&producer_tids[i], NULL, producer_thread, &producer_ids[i]) != 0) {
317             perror("pthread_create producer failed");
318             exit(1);
319         }
320         get_current_time(time_str, sizeof(time_str));
321         printf("[%s] 创建生产者线程%d\n", time_str, producer_ids[i]);
322     }
323
324     // 3. 创建消费者线程
325     pthread_t consumer_tids[CONSUMER_NUM];
326     int consumer_ids[CONSUMER_NUM];
327     for (int i = 0; i < CONSUMER_NUM; i++) {
328         consumer_ids[i] = i + 1;
329         if (pthread_create(&consumer_tids[i], NULL, consumer_thread, &consumer_ids[i]) != 0) {
330             perror("pthread_create consumer failed");
331             exit(1);
332         }
333         get_current_time(time_str, sizeof(time_str));
334         printf("[%s] 创建消费者线程%d\n", time_str, consumer_ids[i]);
335     }
336
337     // 4. 等待所有线程结束
338     for (int i = 0; i < PRODUCER_NUM; i++) {
339         pthread_join(producer_tids[i], NULL);
340     }
341     for (int i = 0; i < CONSUMER_NUM; i++) {
342         pthread_join(consumer_tids[i], NULL);
343     }
344
345     // 5. 释放资源
346     sem_destroy(&empty_sem);
347     sem_destroy(&full_sem);
348     pthread_mutex_destroy(&mutex);
349
350     BUFFER_ST* temp = buffer_head;
351     while (temp != NULL) {
352         BUFFER_ST* next = temp->nextbuf;
353         free(temp);
354         temp = next;
355     }
356
357     get_current_time(time_str, sizeof(time_str));
358     printf("=====\n");
359     printf("实验结束: 共生产%d个产品, 消费%d个产品\n",
360           produced_count, consumed_count);
361     printf("=====\n");
362
363     return 0;
364 }
```


4、编译与运行

```
gcc producer_consumer.c -o producer_consumer -pthread
```

```
chmod +x producer_consumer
```

```
./producer_consumer > producer_consumer_log.txt 2>&1
```

```
LZH2311451@LZH2311451-pc:~$ gcc producer_consumer.c -o producer_consumer -pthread
LZH2311451@LZH2311451-pc:~$ chmod +x producer_consumer
LZH2311451@LZH2311451-pc:~$ ./producer_consumer > producer_consumer_log.txt 2>&1
LZH2311451@LZH2311451-pc:~$
```

将输出重定向到文件中，以便观察。

5、分析结果

首先来看最开始的输出，可以看到各生产和消费线程均能成功创建并投入使用，并且各线程可以竞锁交替工作。

```
producer_consu...
=====
生产者消费者实验：生产者2个，消费者4个，缓冲区大小5，总产品10个
=====
缓冲区初始化完成：共5个节点
[2025-12-31 04:08:11] 创建生产者线程1
[2025-12-31 04:08:11] 创建生产者线程2
[2025-12-31 04:08:11] 创建消费者线程1
[2025-12-31 04:08:11] 创建消费者线程2
[2025-12-31 04:08:11] 创建消费者线程3
[2025-12-31 04:08:11] 创建消费者线程4
[2025-12-31 04:08:11] 生产者2：开始生产，预计耗时1秒
[2025-12-31 04:08:11] 生产者1：开始生产，预计耗时1秒
[2025-12-31 04:08:12] 生产者2：生产产品1完成（校验码：1767125293）
[2025-12-31 04:08:12] 生产者2：准备进入临界区
[2025-12-31 04:08:12] 生产者2：已进入临界区
[2025-12-31 04:08:12] 生产者2：将产品1存入缓冲区1
[2025-12-31 04:08:12] 生产者2：已离开临界区
[2025-12-31 04:08:12] 生产者2：开始生产，预计耗时3秒
[2025-12-31 04:08:12] 消费者1：准备进入临界区
[2025-12-31 04:08:12] 消费者1：已进入临界区
[2025-12-31 04:08:12] 消费者1：从缓冲区1取出产品1（生产者：2，生产时间：2025-12-31 04:08:12）
[2025-12-31 04:08:12] 消费者1：已离开临界区
[2025-12-31 04:08:12] 消费者1：开始消费产品1，预计耗时1秒
[2025-12-31 04:08:12] 生产者1：生产产品2完成（校验码：1767125294）
[2025-12-31 04:08:12] 生产者1：准备进入临界区
[2025-12-31 04:08:12] 生产者1：已进入临界区
[2025-12-31 04:08:12] 生产者1：将产品2存入缓冲区1
[2025-12-31 04:08:12] 生产者1：已离开临界区
[2025-12-31 04:08:12] 生产者1：开始生产，预计耗时3秒
[2025-12-31 04:08:12] 消费者2：准备进入临界区
[2025-12-31 04:08:12] 消费者2：已进入临界区
[2025-12-31 04:08:12] 消费者2：从缓冲区1取出产品2（生产者：1，生产时间：2025-12-31 04:08:12）
[2025-12-31 04:08:12] 消费者2：已离开临界区
[2025-12-31 04:08:12] 消费者2：开始消费产品2，预计耗时2秒
```

再来看最后的一部分输出，可以看到能够顺利的完成规定数量的任务，并且各线程能够正确的有序离场，无忙等待和死锁现象。

观察到一些反直觉现象，如：消费者4已经报告“所有产品已消费完毕，退出线程”，在这之后生产者1却又执行了一系列生产操作（将产品9存入缓冲

区 1)。这是因为代码中，消费者线程每次循环中在判断产品未消费够之后，便会先将全局变量 consumed_count 加 1，相当于预订消费名额，然后等待信号量，被唤醒后再进行消费，而不是在消费后再将 consumed_count 加 1。

所以有可能在生产者 1 和生产者 2 阻塞期间，消费者 1 和消费者 2 已经完成了他们上一个任务并“预订”了最后的产品 9 和产品 10，此时消费者 3 和消费者 4 看到的 consumed_count 便已经是 10，从而退出线程，这时生产者 1 和生产者 2 上 cpu，继续生产剩余的产品。

也就是说“所有产品已消费完毕”实际上是“所有产品已被预订消费”。

```
[2025-12-31 04:08:20] 消费者4：消费产品7完成（校验码：1767125306）
[2025-12-31 04:08:20] 消费者4：所有产品已消费完毕，退出线程
[2025-12-31 04:08:22] 生产者1：生产产品9完成（校验码：1767125311）
[2025-12-31 04:08:22] 生产者1：准备进入临界区
[2025-12-31 04:08:22] 生产者1：已进入临界区
[2025-12-31 04:08:22] 生产者1：将产品9 存入缓冲区1
[2025-12-31 04:08:22] 生产者1：已离开临界区
[2025-12-31 04:08:22] 生产者1：所有产品已生产完毕，退出线程
[2025-12-31 04:08:22] 消费者2：准备进入临界区
[2025-12-31 04:08:22] 消费者2：已进入临界区
[2025-12-31 04:08:22] 消费者2：从缓冲区1 取出产品9（生产者：1，生产时间：2025-12-31 04:08:22）
[2025-12-31 04:08:22] 消费者2：已离开临界区
[2025-12-31 04:08:22] 消费者2：开始消费产品9，预计耗时2秒
[2025-12-31 04:08:22] 生产者2：生产产品10完成（校验码：1767125312）
[2025-12-31 04:08:22] 生产者2：准备进入临界区
[2025-12-31 04:08:22] 生产者2：已进入临界区
[2025-12-31 04:08:22] 生产者2：将产品10 存入缓冲区1
[2025-12-31 04:08:22] 生产者2：已离开临界区
[2025-12-31 04:08:22] 生产者2：所有产品已生产完毕，退出线程
[2025-12-31 04:08:22] 消费者1：准备进入临界区
[2025-12-31 04:08:22] 消费者1：已进入临界区
[2025-12-31 04:08:22] 消费者1：从缓冲区1 取出产品10（生产者：2，生产时间：2025-12-31 04:08:22）
[2025-12-31 04:08:22] 消费者1：已离开临界区
[2025-12-31 04:08:22] 消费者1：开始消费产品10，预计耗时2秒
[2025-12-31 04:08:23] 消费者3：消费产品8完成（校验码：1767125308）
[2025-12-31 04:08:23] 消费者3：所有产品已消费完毕，退出线程
[2025-12-31 04:08:24] 消费者2：消费产品9完成（校验码：1767125311）
[2025-12-31 04:08:24] 消费者2：所有产品已消费完毕，退出线程
[2025-12-31 04:08:24] 消费者1：消费产品10完成（校验码：1767125312）
[2025-12-31 04:08:24] 消费者1：所有产品已消费完毕，退出线程
=====
实验结束：共生产10个产品，消费10个产品
=====
```

三、结论

本次实验成功实现了生产者-消费者问题，练习了多线程环境下的资源同步与互斥。通过信号量与互斥锁的协同使用，遵守“先信号量操作、后互斥锁控制”的逻辑，有效避免了缓冲区溢出、数据竞争及死锁问题，确保 2 个生产者线程与 4 个消费者线程有序协作，完成了 10 个产品的生产与消费全流程。